

Module 7 Capstone Research and Synthesis

Overview and study route

Software Engineering in the Era of AI | Master's year one | Electrical and Information Engineering | Weeks 15 and 16 | 2026 to 2027

The capstone combines the course's engineering and research responsibilities. A convincing result is a reproducible software artifact together with a defensible account of what it does, why it was designed that way, how it was evaluated and where the evidence stops. A polished demonstration alone cannot establish those claims.

Week 15 integrates and freezes the project release. Week 16 contains the independent final examination and project presentations and defence. The project has developed since the Week 4 proposal; these weeks are for synthesis and verification, not for starting an entirely new platform.

Learning outcomes

- Turn the project evaluation question into a reproducible method with a suitable baseline.
- Trace requirements through implementation, tests and reported evidence.
- Separate measured results from interpretation and future work.
- Reproduce a release from its documented environment and data.
- Explain individual contributions and AI assistance honestly.
- Defend a design under an unfamiliar modification or failed case.
- Critically assess emerging software-engineering claims without relying on product popularity.

This module integrates LO1 to LO7, with particular emphasis on LO5 and LO7.

1 A master's project needs a bounded question

A project question should identify a context, intervention, comparator and outcome. "Can AI write software?" is too broad. A tractable question is: for this simulated maintenance reporting task, does retrieval-grounded advice improve human-rated factual support compared with ungrounded generation, while meeting the specified response and cost limits?

Another valid question concerns development rather than runtime AI: does a task context package reduce unit and contract violations when an assistant modifies the energy service, compared with a broad instruction? The project must distinguish these questions because their units of evaluation differ. One evaluates generated application responses; the other evaluates a development workflow.

Keep the scope compatible with the student team's programming capacity. A small reliable artifact with a carefully designed comparison can demonstrate greater depth than a broad application assembled from unexplained frameworks. Explain why the chosen question matters to electrical or information engineering.

2 Minimum viable evidence

Begin with a deterministic baseline and a precise acceptance contract. The baseline establishes what can be achieved without the proposed AI contribution. For energy computation, use the validated Python function. For manual lookup, use a simple authorised retrieval method. For development workflow, use an independently completed or carefully controlled reference task.

The AI component should contribute to a stated need. If a fixed rule answers the question accurately and cheaply, adding a model solely to satisfy an AI theme is not justified. A valid project can demonstrate that the proposed AI feature does not improve the chosen task, provided the experiment and interpretation are sound and the required integration investigation is documented.

Use an evidence matrix to connect claims to artifacts. The matrix should reveal gaps, not merely repeat that everything is complete.

Claim	Required evidence	Example limitation
Energy calculation follows the contract	Independent numerical and invalid-input tests	Does not validate field sensor calibration
Advice uses approved information	Retrieved passages and human-reviewed claim support	Small manual collection
Failures preserve core reporting	Timeout and invalid-output cases	Stub failures do not measure provider availability
Release can be reproduced	Clean-run instructions and observed rerun	Tested on one operating environment
AI workflow improved the task	Comparable tasks and complete time-quality records	Small sample and possible learning effects

3 Design the evaluation before the final run

Define the unit of analysis: a request, a development task, a device session or another meaningful item. State inclusion rules, outcome criteria and failure categories. Choose development and final evaluation cases before tuning. If you revise the task or rubric, version the revision and explain which results use it.

For a small response evaluation, include ordinary questions, boundary or ambiguous questions, missing-evidence questions and adversarial context cases. The distribution should reflect the intended use or be explicitly described as a stress test. Do not combine these populations into one headline score without explaining the weighting.

A suggested project-scale suite is 24 distinct cases, divided into development and final sets, with repeated trials on selected stochastic cases. This is a feasible learning exercise, not a statistical guarantee. More cases may be needed for a substantive reliability claim. If resources limit the suite, report the limitation instead of inflating certainty.

For human grading, define a rubric with examples before rating final outputs. Have two people independently label a subset where possible. Inspect disagreement to determine whether the rubric is ambiguous, evidence is missing or the response is genuinely contestable. Agreement is useful evidence about the grading process but does not itself establish truth.

4 Preserve the raw results

Store task identifiers, input or permitted input reference, configuration, output, grade, failure reason, elapsed time and usage information. Record actual attempts, including failed and abandoned ones according to the predeclared protocol. Keep sensitive information out of the public report and use approved access controls for retained evaluation material.

Do not replace failed runs with successful retries without disclosing the rule. Report first-attempt success separately from success after a retry budget. If a provider outage makes a trial invalid under the protocol, retain the record and explain the exclusion rather than quietly deleting it.

A table of final averages is not enough to reproduce the conclusion. Retain raw observations and the calculation script so another reader can check denominators, exclusions and summaries. The supplied local examples demonstrate recordable deterministic runs; students must collect their own project results.

5 Interpret results with proportionate claims

Separate observation, interpretation and decision. Observation: 18 of 24 selected cases met the rubric. Interpretation: failures clustered in questions requiring two manual passages. Decision: improve retrieval coverage before deployment. These statements have different evidential status.

Compare against the baseline. An AI feature with 75% accepted responses may be useful if the baseline cannot answer the task, or inadequate if a deterministic lookup already achieves near-perfect results. Consider cost, latency, abstention and error consequences alongside the average quality score.

State threats to validity. Construct validity asks whether the measure captures the intended concept; typing speed does not capture overall engineering productivity. Internal validity asks whether another factor explains the difference; task order may create a learning effect. External validity concerns transfer to other users, devices or repositories. Conclusion validity concerns whether the data and analysis support the strength of the inference.

METR's 2026 study-design update is a useful example of revising a method when adoption changes who participates and how time is recorded. [R1] The lesson is to inspect the measurement process, not to assume that a dated estimate applies to every new tool or student cohort.

6 Write the technical report

Use a clear title naming the engineering task. The opening should state the problem, implemented approach, principal measured finding and most important limitation. Write the abstract after the results are known so it cannot promise findings that were never obtained.

A suitable report structure is problem and requirements; related work; architecture and data; implementation; evaluation method; results; discussion and limitations; conclusion; reproducibility and assistance statement. Append detailed test cases or raw records when they would interrupt the argument, but keep the essential evidence in the main text.

Related work should compare approaches, not list tool names. Explain what a source studied, how its setting differs and which idea informs the project. A vendor engineering article may explain a useful implementation pattern; it is not automatically independent evidence of productivity or reliability.

In the results section, label every table with units, denominators and conditions. Identify simulated data explicitly. Do not write "significantly better" as a synonym for a visible difference; use that term only with a defined statistical analysis and its assumptions, or describe the measured difference directly.

The discussion should explain surprising failures and trade-offs. A well-supported negative result is useful. Avoid a conclusion that proposes deployment despite unaddressed failures in the very acceptance criteria chosen earlier.

7 Reproducibility and release packaging

A release should include source code, a documented interpreter and dependency environment, permitted data or a generator, test commands, evaluation configuration, raw results and a report. Include a short entry path such as README instructions that another student can follow without the author's memory.

Separate deterministic reproduction from stochastic repeatability. A saved model response can reproduce the application's parsing and grading, while a fresh provider call may return different content or use an updated backend. Record the visible model identifier, date and configuration, and do not promise bit-for-bit reproduction of a remote model unless the provider and setup actually support it.

Before submission, use a clean copy or fresh environment. Follow the instructions literally, run the tests, run the application and verify the expected output. Record what was actually reproduced and on which environment. Missing local files, absolute paths to the author's machine and undocumented credentials are common reproducibility failures.

Freeze the submitted release in Week 15. The Week 16 demonstration should identify that version. If a correction is required under departmental arrangements, identify the correction and preserve the original submitted state rather than silently replacing evidence.

8 Individual understanding and AI accountability

Each student should explain at least one requirement, one code path, one test oracle, one failure case and one design trade-off. Shared code does not remove individual responsibility. A contribution statement should describe observable work such as implementing validation, designing cases, reviewing changes or analysing results.

An AI assistance record states which task was delegated, the relevant tool and version information, what was accepted or rejected and how it was checked. Do not fabricate a prompt history for a supplied example. If you used an instructor-provided candidate, say so. Do not claim that a tool performed an external test when only a stub was executed.

The oral defence is a short evidence check, not a performance contest. Explain an unfamiliar change: if signed power becomes valid, which assumptions, tests and report labels change? If a manual revision removes a procedure, which cache and evaluation records must be updated? Such questions reveal whether the student understands the system beyond its successful demonstration.

9 Assessment rubric and standards

The course retains the 20% project, 10% documentation and reflection, and 70% final examination structure. The following descriptors guide judgement within those allocations; they do not create new marks.

Dimension	Strong evidence	Weak evidence
Requirements	Precise domain, units and acceptance criteria	Broad feature list with unstated assumptions
Design	Alternatives and justified boundaries	Diagram unrelated to the implemented code
Implementation	Working, readable and explainable	Copied fragments the student cannot trace
Verification	Independent cases and meaningful failure tests	Only a successful demonstration
Evaluation	Baseline, raw records and qualified conclusions	Unsupported headline success rate
Reproducibility	Clean rerun from documented artifacts	Depends on hidden local state
Reflection	Evidence-based account of limitations and AI use	Generic praise of tools or fabricated interactions

Within the main project, requirements receive 4 course percentage points, architecture 3, implementation 5, verification 5 and delivery 3. Documentation and reflection allocate 4 to technical documentation, 2 to AI accountability, 3 to research evaluation and reflection, and 1 to individual defence. The outline is the authority for these allocations.

10 Examination synthesis

The independent final examination uses a two-hour paper marked out of 100 and scaled to 70% of the course. Prepare to reason from supplied code, data and scenarios. Product-specific command memorisation is less important than explaining contracts, failure paths and evidence.

The blueprint allocates 10 marks to Module 1, 20 each to Modules 2, 3 and 4, 10 to Module 5, 15 to Module 6 and 5 to Module 7. An integrated question can span modules, but its marks are counted once. The department should publish permitted reference and calculator arrangements in advance.

Study by tracing unfamiliar code, designing a counterexample, calculating a metric and defending a trade-off. Reading model answers without attempting the questions first can create false familiarity. Use the answer controls in the HTML notes to keep your initial attempt independent.

11 Evaluate future trends without rewriting the foundations

New agents, tool protocols, execution environments and model capabilities will continue to appear. Evaluate a trend through the engineering task it changes. Does it improve accepted outcomes, reduce review burden, broaden authority, alter data exposure or introduce a new dependency? Which evidence would justify adopting it?

Distinguish a capability benchmark from workplace productivity and from application reliability. Benchmark tasks may differ from a student's codebase, permissions and review standards. A tool that succeeds on long tasks may still need careful requirements and an evaluator that cannot be bypassed.

Maintain a short update register with the source date, relevant claim, affected course activity and proposed verification. The 2026 to 2027 course pack is based on sources checked in September 2026; later changes should be incorporated after checking rather than predicted as established facts.

Weekly laboratories and assessment sessions

Week 15 lab Reproduce and submit

- Minutes 0 to 15: use a clean copy and follow the release instructions.
- Minutes 15 to 30: execute deterministic tests and the documented demonstration.
- Minutes 30 to 45: verify evaluation denominators, source versions and assistance records.
- Minutes 45 to 55: check the evidence matrix for unsupported claims.
- Minutes 55 to 60: freeze and submit the release, report and required supporting artifacts.

Preparation includes completing the implementation and collecting results before the lab. The lab validates readiness; it is not enough time to perform an entire project experiment.

Week 16 examination and defence

The two-hour lecture allocation becomes the independent final examination. The one-hour laboratory allocation becomes presentations and individual defence. Use short group demonstrations and targeted individual questions. For a large cohort, the instructor should organise parallel examiner panels and pre-submitted demonstrations within the allocated period.

Each defence should identify the submitted version, demonstrate a requirement, explain a failed case and discuss a proposed change. Do not award credit merely for polished slides or an expensive AI subscription. Assess the student's understanding and the project's evidence.

Practice questions

Question 1 Research question

Improve "AI improves maintenance" into a bounded question suitable for this course.

Model answer. For example: on a specified set of synthetic maintenance queries and approved manual revisions, does retrieval-grounded generation improve human-rated factual support compared with generation without retrieval, under the same latency and cost limits? State the population, comparator and outcome.

Question 2 Baseline

Why include a deterministic baseline when evaluating an AI feature?

Model answer. It shows what the task can achieve without the proposed AI contribution and supplies a meaningful comparison. The added complexity and cost need evidence of useful benefit. A baseline may also provide a fallback.

Question 3 Negative result

The AI feature does not outperform a simple lookup. Can the project still demonstrate master's-level achievement?

Model answer. Yes, if the artifact, comparison and interpretation are rigorous and the required investigation is completed. Explain why the result occurred, the evidence limits and the justified design decision. A positive claim should not be manufactured.

Question 4 Raw records

Why are final averages insufficient for reproducibility?

Model answer. They hide task-level failures, exclusions, denominators and variability. Raw observations and calculation steps let a reader verify the summary and investigate patterns, subject to data-access restrictions.

Question 5 Learning effect

A student solves a task manually, then repeats the same task with AI and finishes faster. What confound exists?

Model answer. The student already understands the task and solution. Use matched different tasks and counterbalanced order where possible, and report remaining differences. The time change cannot be attributed entirely to AI.

Question 6 Claim scope

Rewrite "our system is reliable" when the evidence is 18 accepted responses out of 24 selected classroom cases.

Model answer. For example: the system met the stated rubric on 18 of 24 selected classroom cases, an observed acceptance rate of 75%; the small, selected set does not establish field reliability. Report failure categories and evaluation conditions.

Question 7 Replay

What does replaying saved provider responses establish, and what does it not establish?

Model answer. It can reproduce parsing, validation, grading and fallback behaviour on those responses. It does not measure a current provider's availability, live latency or fresh response quality.

Question 8 Reproducible release

List five artifacts a peer needs to rerun the project.

Model answer. Source release, environment and dependency record, permitted data or generator, run and test instructions, and evaluation configuration with raw results. Include credentials setup without exposing secrets if live access is optional or required.

Question 9 Individual contribution

A student wrote few lines but designed the independent tests and found a unit error. Is code volume an adequate contribution measure?

Model answer. No. Test design, review and defect diagnosis are substantive engineering contributions. Assess observable work, its effect and the student's ability to explain it, rather than counting generated lines.

Question 10 Changed requirement

The project must now accept negative net power to represent export. What should change before implementation?

Model answer. Revise the domain, physical interpretation and report labels; define whether results represent net energy or separate import and export; update acceptance and invalid-input cases; review downstream thresholds and documentation. Then implement and verify the agreed contract.

Question 11 Benchmark transfer

A new agent scores highly on public coding tasks. What additional evidence is needed before adopting it for this course's project?

Model answer. Evaluate representative project tasks under the intended context, permissions, quality rubric and budget. Measure review burden, failures and compatibility. Public tasks may differ in language, repository knowledge and acceptance standards.

Question 12 Integrated examination case

A service reports correct energy but produces advice from an old manual, logs an API key and has no reproducible version record. Identify three distinct deficiencies.

Model answer. The advice has a provenance and cache or retrieval-version problem. The log exposes a credential and requires containment and secret-handling correction. Missing release evidence prevents reliable reproduction and diagnosis. Correct arithmetic addresses none of these separate defects.

Final revision checklist

Trace one claim to its raw evidence. Reproduce the submitted version. Explain the baseline and the independent test set. Distinguish observation from interpretation. State one validity threat and one remaining risk. Defend an unfamiliar requirement change. Describe AI assistance accurately and identify your own contribution.

Sources and further reading

- [R1] METR. We are Changing our Developer Productivity Experiment Design, 24 February 2026.
<https://metr.org/blog/2026-02-24-uplift-update/>
- DORA. State of AI-assisted Software Development 2025.
<https://dora.dev/research/2025/dora-report/>
- ACM. Artifact Review and Badging.
<https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- Further reading: Winters, Manshreck and Wright, Software Engineering at Google, chapters on knowledge sharing, testing and engineering productivity.

References checked 15 September 2026. Proposed evaluation sizes and scenarios are teaching guidance, not actual project results. Students must populate their reports with their own observed evidence.